Brute Force Cyber Attack Detection Simulator

Final Project Report

Data Structures and Algorithms — Sliding Window Attack Detection

Authors	Glory Mturi Elias, Cem Suay Asildo■an, Deniz Doga
Course	SEN2211 Data Structures I / SEN2212 Data Structures II
Project Title	Brute Force Cyber Attack Detection Simulator
Supervisor	[Supervisor Name]
Date	2025
Language	Java (OpenJDK 21)

Abstract

*This report presents the design, implementation, and analysis of a Brute Force Cyber Attack Detection Simulator built in Java. The system monitors failed login attempts per IP address using a sliding window algorithm backed by a **HashMap** and a **LinkedList** queue. The project demonstrates the direct application of data structures studied in SEN2211 and SEN2212 to a real-world cybersecurity problem, achieving O(1) amortised detection time per attempt with bounded memory usage.*

Table of Contents

1. Introduction

A **brute force attack** is one of the most common and persistent threats in cybersecurity. In this attack, an adversary systematically attempts large numbers of credential combinations — usernames and passwords, PINs, or cryptographic keys — until access is gained. Despite requiring no sophisticated knowledge of the target system, brute force attacks succeed against systems that lack rate-limiting or detection mechanisms.

This project addresses the problem of *real-time* brute force attack detection at the network level. The core objective is to build a Java-based simulator that monitors failed login attempts per IP address, raises alerts when suspicious patterns are detected, and blocks offending sources — all within tight time and space complexity bounds.

The project directly applies data structures and algorithms studied in **SEN2211** (Data Structures I: linked lists, queues) and **SEN2212** (Data Structures II: hash-based structures), demonstrating their relevance to practical software security problems.

2. Background and Related Concepts

2.1 Brute Force Attacks

Brute force attacks exploit authentication endpoints by trying many credential combinations in rapid succession. They are detectable by the unusually high rate of failed login attempts from a single source within a short time window. Production systems defend against them using account lockouts, CAPTCHA challenges, IP-based rate limiting, and intrusion detection systems (IDS).

2.2 Rate Limiting and Sliding Window

Rate limiting restricts how many requests a client can make within a time period. The **sliding window** algorithm is one of the most accurate approaches: rather than resetting a counter at fixed intervals (which can miss bursts that straddle a boundary), it maintains a rolling log of event timestamps and evaluates only those within the last *T* milliseconds at any given moment. This gives exact, continuous measurement of request frequency.

2.3 Relevant Data Structures

Two fundamental structures drive the detection engine:

- **HashMap<String, LinkedList<Long>>** — maps each IP address to its attempt history. Chosen for O(1) average-case lookup and insertion.
- **LinkedList<Long> as a FIFO Queue** — stores ordered Unix timestamps for each IP. Chosen for O(1) head removal (expired entries) and tail insertion (new attempts).
- **HashSet<String>** — tracks blocked IPs for O(1) membership testing.

3. System Design and Data Structure Selection

The detection engine is built around a single compound data structure:

```
HashMap<String, LinkedList<Long>> loginAttempts
```

Each key is an IP address string. Each value is a LinkedList of timestamps representing recent failed login attempts from that IP. This design was chosen over alternatives for the following reasons:

Structure	Lookup	Insert	Ordered?	Used Here?
HashMap	O(1) avg	O(1) avg	No	Yes — primary map
TreeMap	O(log n)	O(log n)	Yes (key)	No — too slow
Array/List	O(n)	O(1)	Yes	No — O(n) lookup
LinkedList (Queue)	O(1) head/tail	O(1)	Yes (FIFO)	Yes — per-IP queue
ArrayDeque	O(1) head/tail	O(1)*	Yes (FIFO)	Alternative option

Table 1: Data structure comparison for the detection engine

4. Algorithm Description

The core detection method `logAttempt(String ip)` executes four steps on every incoming failed login attempt:

Step 1 — HashMap Insert (O(1))

If the IP has not been seen before, a new empty LinkedList is created and associated with it using putIfAbsent(). This is idempotent — calling it again for an existing IP has no effect.

Step 2 — Timestamp Append (O(1))

The current system time (System.currentTimeMillis()) is appended to the tail of the LinkedList using addLast(). Since LinkedList maintains a direct tail pointer, this is O(1) regardless of list size.

Step 3 — Window Expiry (amortised O(1))

A while loop inspects the head of the list. Any timestamp older than TIME_WINDOW milliseconds (default: 10,000 ms) is removed via removeFirst(). Each timestamp is added once and removed at most once over its lifetime, giving amortised O(1) per call.

Step 4 — Threshold Check (O(1))

The list's size() is compared against THRESHOLD (default: 5). Since LinkedList maintains an internal element count, size() is O(1). If the count meets or exceeds the threshold, an alert is triggered and the IP is added to the blockedIPs HashSet.

Java implementation of the core algorithm:

```
loginAttempts.putIfAbsent(ip, new LinkedList<>()); // Step 1
LinkedList<Long> ts = loginAttempts.get(ip);
ts.addLast(System.currentTimeMillis()); // Step 2
while (!ts.isEmpty() &&
now - ts.peekFirst() > TIME_WINDOW) // Step 3
```

```
ts.removeFirst();
if (ts.size() >= THRESHOLD) { /* trigger alert */ } // Step 4
```

5. Complexity Analysis

5.1 Time Complexity

Operation	Average Case	Worst Case	Notes
HashMap get / putIfAbsent	O(1)	O(n)	Worst case: hash collision chain
LinkedList addLast	O(1)	O(1)	Direct tail pointer
LinkedList peekFirst	O(1)	O(1)	Direct head pointer
LinkedList removeFirst	O(1)	O(1)	Direct head pointer
LinkedList size()	O(1)	O(1)	Internal counter maintained
Window expiry loop	O(k) per call	O(k) per call	Amortised O(1): each entry removed once
Full logAttempt() call	**O(1) amortised**	**O(n+k)**	n=IPs (collision), k=expired entries

Table 2: Time complexity of all detection engine operations

5.2 Space Complexity

The total memory usage of the detection engine is **O(I × T)**, where:

- **I** = number of unique IP addresses currently being tracked
- **T** = maximum number of timestamps stored per IP, which is bounded by THRESHOLD (default 5) — once an IP is blocked, no further timestamps are admitted

In practice I and T are both small, making the memory footprint negligible. The HashMap itself maintains an array of bucket pointers with a default load factor of 0.75, resizing when 75% of buckets are occupied.

6. Implementation

The detection engine was implemented as a single Java class `BruteForceDetector` using only the Java standard library (no external dependencies). Key implementation details:

- **Configurable parameters:** THRESHOLD and TIME_WINDOW are read at runtime, allowing live reconfiguration without restarting the system.
- **Status escalation:** Each IP progresses through four states — CLEAN (0–59% of threshold), WATCHING (60–99%), ATTACK (at threshold), BLOCKED (post-alert).
- **Non-blocking simulation:** The original Thread.sleep() loop from main() was replaced by a javax.swing.Timer / JavaFX Timeline to keep the UI responsive.

• **Passive window expiry:** A background timer calls refreshTable() every second, pruning expired timestamps from all tracked IPs even when no new attempts arrive — ensuring BLOCKED IPs eventually revert to CLEAN if the attack ceases.

• **Two GUI implementations:** Java Swing (dark-theme dashboard) and JavaFX (CSS-styled cybersecurity dashboard with live bar chart), both sharing the same unchanged detection core.

7. Testing and Results

The system was validated against six test scenarios:

Scenario	Input	Expected	Result
Normal usage	1–2 attempts from multiple IPs	All CLEAN	**PASS**
Escalation	6 attempts at 500 ms intervals (threshold=5, window=10s)	CLEAN→WATCHING→ATTACK→BLOCKED	**PASS**
Window expiry	Attack then silence for 10+ seconds	Status reverts to CLEAN	**PASS**
Mixed traffic	Attacker + legitimate user simultaneously	Only attacker blocked	**PASS**
Threshold change	Change threshold 5→10 mid-session	Alert delayed to 10th attempt	**PASS**
Rapid-fire burst	12 attempts in under 2 seconds	Blocked after 5th, remaining logged as BLOCKED	**PASS**

Table 3: Test scenarios and outcomes

All six test scenarios produced the expected outputs, confirming the correctness of the sliding window algorithm, the accuracy of the HashMap-based IP tracking, and the reliability of the status escalation and expiry logic.

8. Conclusion

This project successfully demonstrated that two foundational data structures — **HashMap** and **LinkedList** — are directly sufficient to solve a real-world cybersecurity problem. The sliding window algorithm provides temporal accuracy that simpler fixed-counter approaches cannot match, operating in **O(1) amortised time** per event with **O(I × T)** space — both highly favourable for deployment in high-throughput environments.

Key lessons reinforced by the project:

• The **HashMap** is the optimal associative container for key-value access when average-case O(1) performance is required and key ordering is not needed.

- The **Queue abstraction** (FIFO LinkedList) is a natural model for time-ordered event streams where old events expire from the front and new ones arrive at the back.
- **Amortised analysis** is essential for correctly evaluating the cost of the expiry loop — a naive worst-case analysis would overestimate the cost significantly.
- Separating **business logic from the presentation layer** (same detection class reused across Swing and JavaFX GUIs) is a fundamental software engineering principle with tangible practical benefits.

Future extensions include: persistent storage for cross-session detection history; adaptive thresholds based on baseline traffic; CIDR subnet matching for distributed attacks; and live packet capture integration via Pcap4J.

9. References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

[2] Liang, Y. D. (2014). *Introduction to Java Programming, Comprehensive Version* (10th ed.). Pearson.

[3] Oracle Corporation. (2023). *Java SE 21 API Documentation: HashMap, LinkedList, HashSet*. https://docs.oracle.com/en/java/

[4] McGraw, G. (2006). *Software Security: Building Security In*. Addison-Wesley.

[5] OWASP Foundation. (2023). *Testing for Brute Force (OTG-AUTHN-003)*. https://owasp.org/www-project-web-security-testing-guide/

[6] Goodrich, M. T., & Tamassia, R. (2015). *Data Structures and Algorithms in Java* (6th ed.). Wiley.